

Project Calculator

Arbitrary precision arithmetic Library

May 2025

Directory Structure

```
project_java/  
  build.xml  
  src/  
    arbitraryarithmetic/  
      AInteger.java  
      AFloat.java  
      MyInfArith.java  
  dist/ aarithmetic.jar  
  run_arith.py  
  report.pdf  
  README.md
```

Introduction

This report describes the design section, which describes the design decisions that I made and also include the class diagrams in UML format.

Team Details

- Name: Petei Divya Poorna Chand
- Roll Number: CS24BTECH11048

Design

The central idea of this project is to perform arithmetic operations on Integers and Floating-Point numbers without using libraries Bignumbers or any other built-in types

- **AInteger**:Handles arbitrarily large integers, including sign management.
- **AFloat**: Supports floating-point arithmetic by separating integer and decimal parts and using **AInteger** for computations.

Design Decisions

This section explains the key design decisions made during the development of the Arbitrary Precision Arithmetic Library.

1. Object-Oriented Approach

We adopted an object-oriented programming (OOP) paradigm to structure the library. Two main classes, `AInteger` and `AFloat`, were created to encapsulate the behavior and data of arbitrary-precision integers and floating-point numbers, respectively.

2. String-Based Internal Representation

To support numbers of arbitrary size and precision, we chose to store numeric values as `String` objects. This allowed us to overcome the limitations of built-in Java data types like `int`, `long`, or `double`, and manually implement arithmetic logic character by character.

3. Manual Arithmetic Implementation

All arithmetic operations ($+$, $-$, $\times$, $\div$) were implemented manually to simulate how operations are performed on paper:

- Addition and subtraction follow digit-wise traversal with carry/borrow handling.
- Multiplication uses column-based manual multiplication.
- Division follows the long division method, including repeated subtraction and decimal handling.

This design ensures high precision and full control over the operations.

4. Separation of Integer and Floating-Point Logic

The decision to use separate classes for `AInteger` and `AFloat` ensures clarity in handling integer-only logic versus floating-point arithmetic:

- `AInteger` handles whole numbers and is optimized for base operations.
- `AFloat` wraps two `AInteger` objects (for integer and fractional parts) and manages decimal point alignment.

5. Command Line Interface (CLI)

We provided a CLI through the `MyInfArith` class. This allows users to perform operations directly from the terminal, taking two number strings and an operator as input and printing the result to the console.

6. Ant-Based Build System

To streamline compilation and packaging, we used Apache Ant to define a build process via `build.xml`. This makes the project easy to build and test with a single command. `aarithmetic.jar` file.

7. Decoupling Testing and Execution

Testing and execution are decoupled using an external Python script. The script builds the project and executes the JAR file with user inputs, mimicking test case runs in a controlled environment.

8. Precision Control

To limit performance while preserving accuracy, we designed the system to support up to 30 decimal digits of precision for floating-point operations. This level of precision is adequate for most use cases and balances speed with accuracy.

implementation

AInteger class

The `AIInteger` class handles arbitrary-precision integers. All numbers are represented as strings to avoid Java's built-in limits. Operations supported include:

- `add(AInteger other)` – Adds two `AIIntegers` using string-based digit-wise addition, Handling carry and sign.
- `subtract(AInteger other)` – Performs subtraction, handling borrowing and sign.
- `multiply(AInteger other)` – Repeated addition of one number using `AIInteger.add(Other)`.
- `divide(AInteger other)` – Repeated subtraction of one number from other using `AIInteger.subtract(other)` and counts repetitions

AFloat Class

The `AFloat` class handles floating-point numbers using `AIInteger` for the integer parts. It supports:

- stores index of decimal point and makes floating point numbers into integers by removing decimal points
- uses `AIInteger` for operations and add the decimal point at required place based on operation
- Manual multiplication and division with precision up to 30 decimal digits.
- sign handling

Command-Line Interface:MyInfArith Class

This class acts as the main interface for users. It parses command-line arguments and implements the appropriate arithmetic operations.

Functionality

- Accepts input in the format: `java MyInfArith type operation num1 num2`
- Determines whether the operands are integers or floating-point based on the presence of a decimal point.
- Calls the corresponding method from `AIInteger` or `AFloat` for the requested operation.
- Outputs the result.

Creating a jar file using ant

I used Apache Ant, a Java-based build automation tool, to compile our source code and package it into a JAR file named `aarithmetic.jar`. This approach allows for reproducible builds and eliminates manual compilation and packaging.

Buildfile : build.xml

The build configuration is defined in a file named `build.xml`. This file specifies the project structure, compilation target, and JAR creation logic.

How to Build

To build and generate the JAR file, simply run:

```
ant jar
```

in terminal This will:

- Compile all Java files under `src/arbitraryarithmetic/`
- Place compiled `.class` files into `target/`
- Create a JAR file `aarithmetic.jar` with `MyInfArith` as the main class

Advantages

Using Ant simplifies the build process by:

- Automating compilation and packaging
- Ensuring consistent builds
- Making the project more portable and easier to test

python code to run the main program

To reduce difficulty in the the development and execution process, we created a Python script that automates compiling the Java source files and running the resulting executable JAR file. This script significantly reduces manual effort and ensures a consistent build environment for testing and demonstration purposes.

Functionality

The Python script performs the following actions:

1. It compiles all the Java files in the `src/arbitraryarithmetic/` directory using the `javac` compiler.
2. It packages the compiled `.class` files into a JAR named `aarithmetic.jar`, setting `MyInfArith` as the main class.
3. It executes the JAR file with user-provided arguments (`type,operator,num1, num2`) and prints the output.

Usage

Command-line format:

```
python3 run_arithmetic.py <type> <operator> <num1> <num2>
```

Class Diagram (UML)

AInteger
- value : String
+ AInteger() + AInteger(value : String) + copy(other : AInteger) : AInteger + parse(s : String) : AFloat + toString() : String + add(other : AInteger) : AInteger + subtract(other : AInteger) : AInteger + multiply(other : AInteger) : AInteger + divide(other : AInteger) : AInteger + compare(String a,String b) : int

AFloat
- value : String
+ AFloat() + AFloat(value : String) + copy(other : AFloat) : AFloat + parse(s : String) : AFloat + toString() : String + add(other : AFloat) : AFloat + subtract(other : AFloat) : AFloat + multiply(other : AFloat) : AFloat + divide(other : AFloat) : AFloat + compare(String a,String b) : int

Limitations

1. Manual Long Division is Time-Consuming:

The `divide` methods in both `AInteger` and `AFloat` use a manual, long-division-style approach implemented through string manipulation. As a result, the execution time increases significantly for large input values.

2. Fixed Precision of 30 Decimal Digits:

The `AFloat` class supports up to 30 digits after the decimal point. Any computation requiring higher precision will be truncated or rounded beyond this limit.